

**Name:** Eoin Geoghegan

**Student No:** 20036009

### **Compound Predefined Workouts Feature.**

#### **How it was done:**

I started by working on the old files for predefined workouts which used to be four views, one for each type (upper, lower, full body and core). Instead of this I deleted these files and put them all together in one file in `models/predefined-workouts.js`. Here there is a list of the hardcoded predefined workouts by type: upper, lower, full-body and core all containing their own exercises with set values. There is an export function that looks for the type of workout, for example if upper body workout is in the URL, then it will render the upper body workout.

Next I worked on the new view and the getting the track button working, which will look the same in the browser but using different code to display it. I also added the route for the track button.

At the beginning the values for weight, sets and reps were hardcoded so I changed this to show an empty field for these. Now the user can enter their own values. When the user enters the values and presses track, it uses the route POST to - `/predefined/(the type of workout)/trackworkout` and calls the controller `trackWorkout` method.

I updated the controller with two methods, one for showing the data(index) and the other for tracking the workouts. This looks for the users' credentials and the workout type found in the URL. Then there is a for loop to allow the data to be entered and updated, after this the workout is then saved to the `trackerStore` using the method in the store.

I updated the MongoDB and JSON store `addTrackedWorkout` method then, which now looks for a workout with an ID or a predefined workout without an ID. This allows either workout to be saved using the same method it as before.

## **Modelling**

### **User Stories**

#### **Gym User -**

The gym user is the person using the application. The user is someone who goes to the gym and wants to keep track of their workouts. To start the user should be able to register to the app, then login using the information they provided. When logged in, the user should then see their own personal dashboard.

For new users there should be predefined workouts already visible in the dashboard for examples of how the workouts can be viewed. The user could use these for getting started and they also demonstrate how to use the app. When the predefined workouts are opened the user should see a list of exercises relating to the workout they opened, empty fields to manually enter data for weights/sets/reps and a track button. If the user wants to use this template for their workouts they should be able to do so by pressing the track button.

They would like the application to allow them to create their own workout plan also which can be saved and displayed in their dashboard. The user should then be able to open the workout they created, they should be able to add exercises with the amount of weight/sets/reps for each exercise. When the user fills out their exercises, they should be able to see what they have entered and then have the option to track it when they are finished exercising, saving and displaying the completed workout. This should also be saved with a timestamp to see when workouts are done. Everything entered by the user should be able to be edited/updated and deleted like the workout title, the exercises entered, the values for sets/weight/reps and the tracked workouts.

#### **Features to be developed from the User Story**

- To be able to register
- Login with the registered information
- When logged to view a personal Dashboard
- In dashboard to show Predefined Workouts Cards
- Predefined Workouts to show exercises relating to the workout type with a track button
- Custom Workouts to save and display in dashboard
- Add exercises to workout and have a track workout button
- Data saved in Database for users, workouts, exercises and tracked Workouts
- Edit/update and delete workouts/exercises + values

## Data Model

From the user story I will need to make a model section to store information for User, Predefined Workouts, Custom workouts, exercises and tracked workouts. This information will be saved within a store in the app which are methods handling the information for each type. The way in which I will develop the app to save this information will be in memory storage at the beginning, then move onto a JSON database and then finally MongoDB which will be setup with Atlas so it can be online.

**User Store-** This will need to store information such as name – first and last, email, and a password. The user should then have their own ID also. This store will have methods for getting the information for all the users, adding a user giving them an ID, deleting a user by their ID, deleting all the users and finding a user by email or ID.

**Predefined Workout** – A predefined workout model will need to store the title of the workout and exercises + values entered by the user for each workout type. The predefined workout might not have an ID as will be hardcoded data. Data entered by the user should be able to be stored from here also and saved to a new page with the workout title and timestamp. Predefined workouts will be hardcoded information for the title of the workout, the name of exercises and the equipment for the workout. It will use the trackedworkouts addTrackedWorkout method for storing workouts.

**Workout** – A workout model will need to save information such as the workout title, have a workout ID and must relate to the user ID. The workout will have exercises within also. The workout title should be able to be edited/deleted so the store will need to be able to handle this. This store will have methods for adding a workout, getting all workouts, deleting and updating workout title

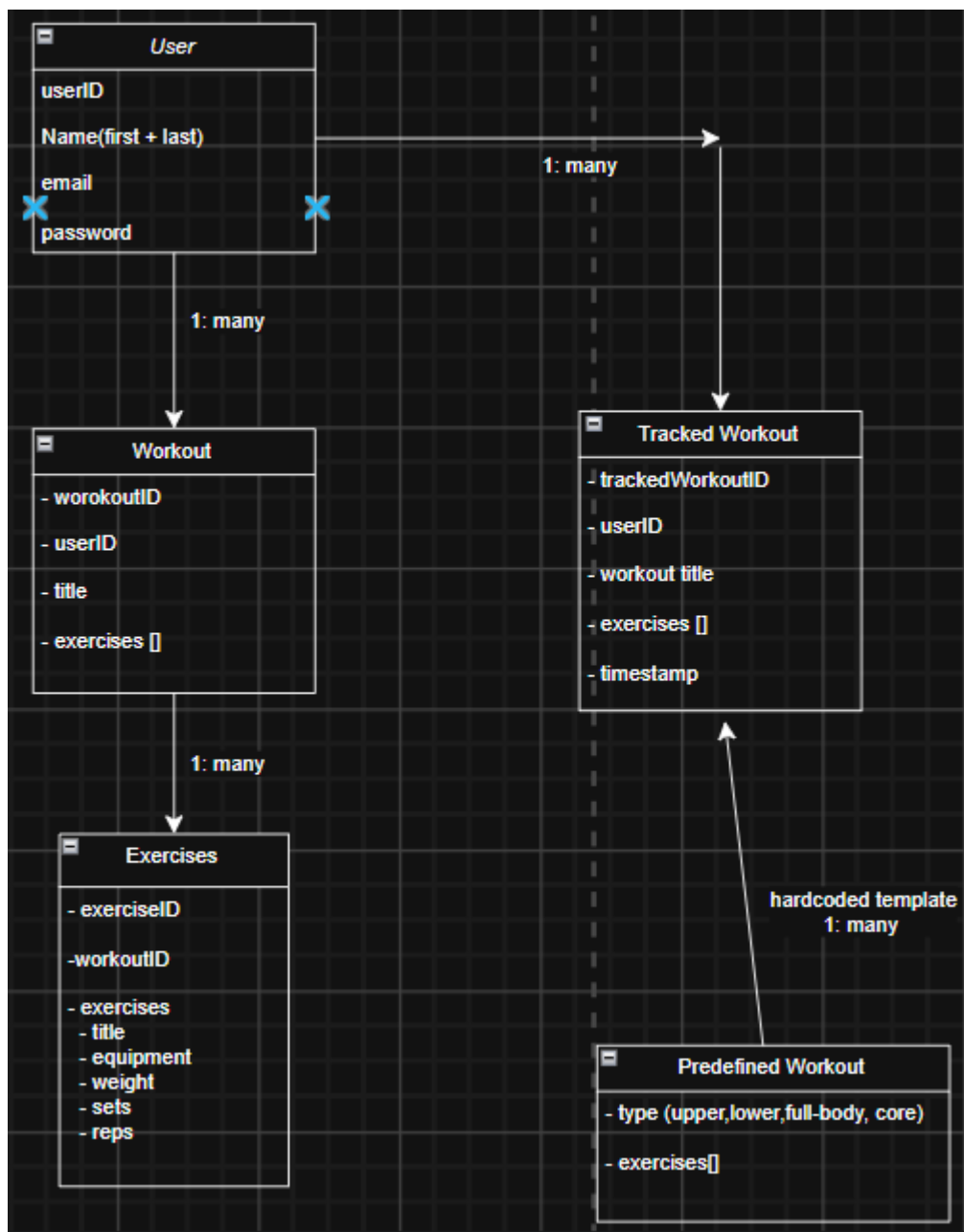
**Exercises-** Exercises needs to store data like exercises by name, equipment used, weight amount, sets amount and reps amount. This data should be stored a list after entering it. The stored entries then should be able to be saved to another page with the workout title, and timestamp and the exercises completed. Exercises will need an exerciseID and the WorkoutID that its related too. Exercise Store will handle adding exercises, getting info for all the exercises that are added, deleting exercises, getting exercises by ID, by workoutID and updating exercises.

**Tracked Workouts** – This should store the workout title from custom/predefined workouts and the exercises from these too. It will need a trackedWorkout ID. The store will have methods for adding a trackedWorkout – copying the exercises from the workout, deleting the tracked workout and getting the tracked workout by ID and userID. This store will also have a method to allow data to be updated.

## Relationship between the models

- Compound app can have many users
- Users can have many workouts
- User can have many tracked workouts
- Custom workouts can have many exercises
- TrackedWorkouts saves copies of exercises from a workout
- TrackedWorkouts can have many predefined workouts

Figure 1.) Relationship Data model diagram



## System Model

This App will be built using Node.js sever and Hapi framework. This will allow for the app to be developed locally, using multiple files with code which will all communicate with each other. It's easier for developing features. Hapi uses plugins to allow the app to be visible in the browser, for authentication and sessions and handling static images. It will use a Model/View/Controller/Route structure which will determine how the information is moved around within the app. The main information will be then stored in store models for User, Workout, Exercises, predefined Workouts and tracked workouts.

**Views-** This is how the data will be seen in the browser. The controller will be responsible for the data that will populate the Views. The views will be handled using handlebars and Hapi Vision. In the views then Bulma will be used for styling to make the pages look better.

**Controller-** The controller will be responsible for receiving information from forms, sending this information to the store, retrieving information back from the store and deciding what information to show in the browser.

**Routes-** The routes will take POST and GET HTTP requests from forms including URL paths, then call the controllers methods depending on what information is being requested.

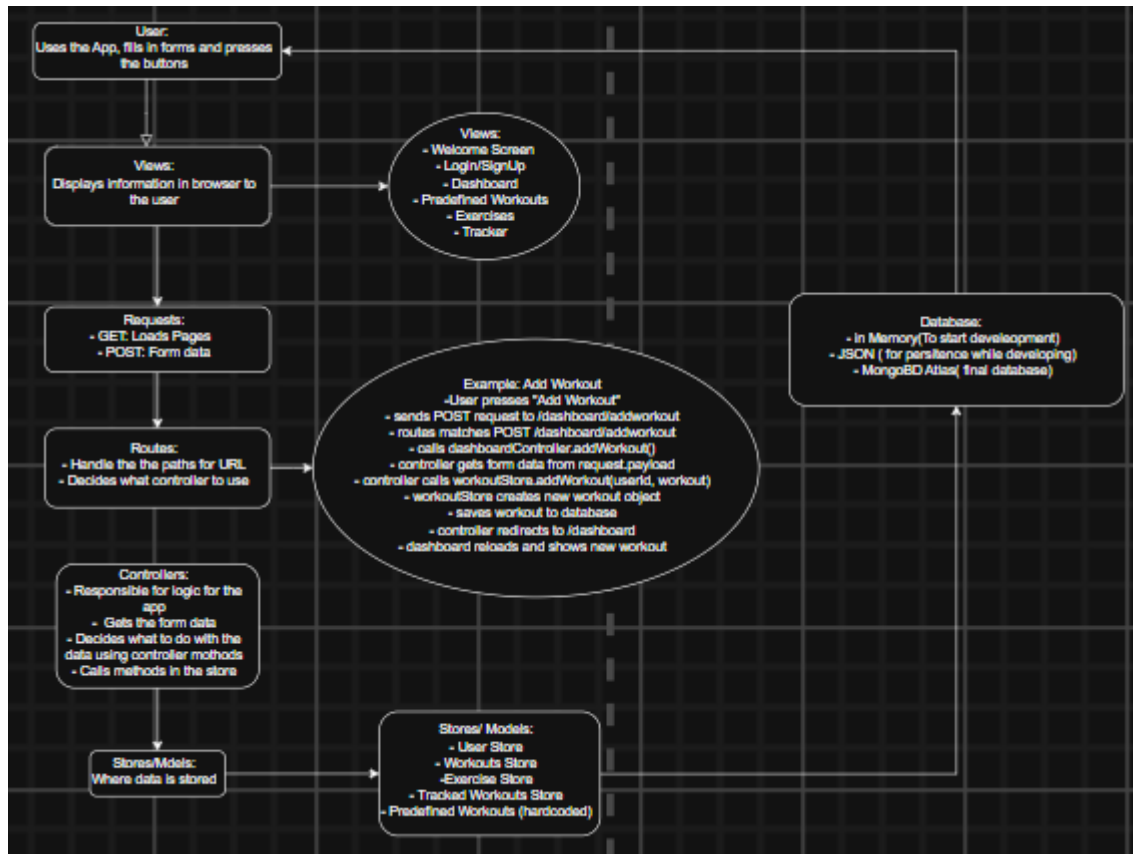
**Models-** As described above will store the information entered by the user in a database. It will begin with in Memory storage, then move to JSON for persistence and end with MongoDB atlas.

### How it communicates:

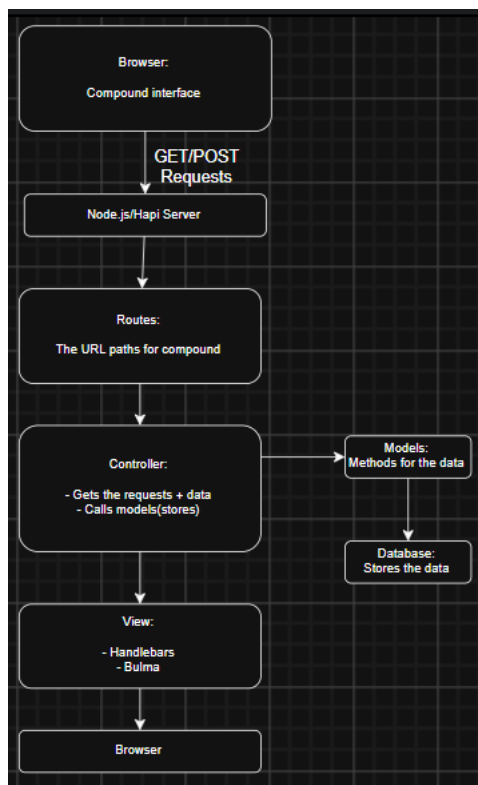
It uses POST requests to submit data for the app and GET requests for loading the data in the browser. An example of how the features work for the entire app is like this: The user will be shown a welcome screen where they have two buttons in a navbar. One for signing up and one for logging in. If the user presses the signup button then this adds /signup to the path and renders the sign-up view. Here the user is shown a login form, where they can enter their data for name, email and password to be stored in the database. When the data is entered and the submit button is pressed this sends a POST request to the routes which looks for the register route in this example. This register route will then call the accounts controllers sign-up method. The accounts controller is then responsible for requesting the information entered by the user, it then sends this information to the UserStores - addUser method. Here the information received is stored in the database and given an ID also. This registers the user to the app and then then controller redirects the user to the login page.

The next step is for the user to enter their saved details into the new login form they are shown. Here when the data is entered the credentials are checked and if successful logs in successfully. The submit button is pressed then the same steps are followed for signing but to different routes/controllers and models in the app designed to handle these requests.

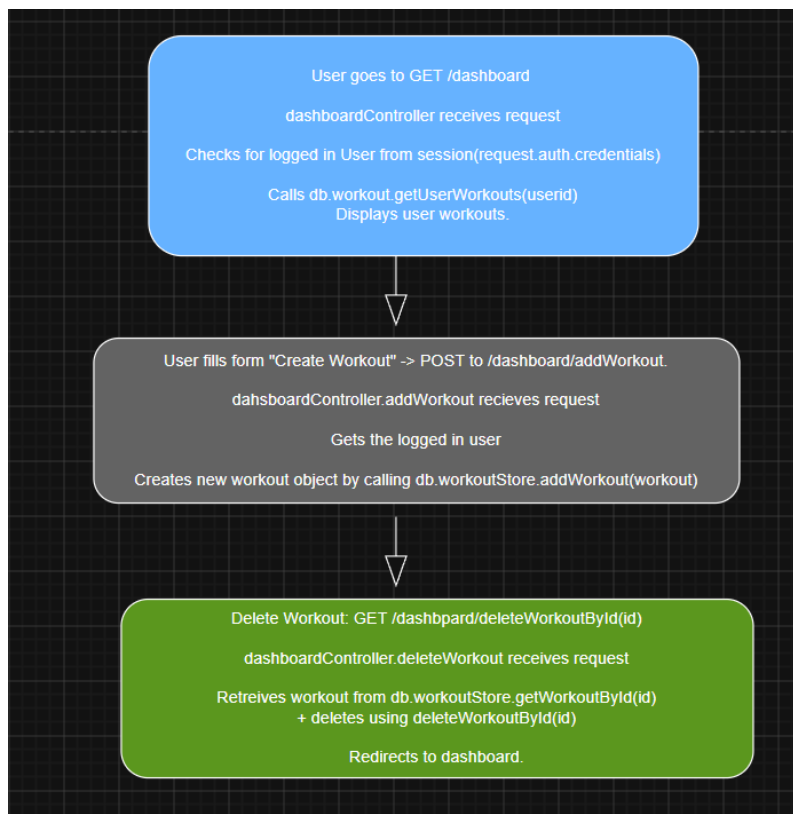
**Figure 2.)** Diagram of MVC for how the app will communicate



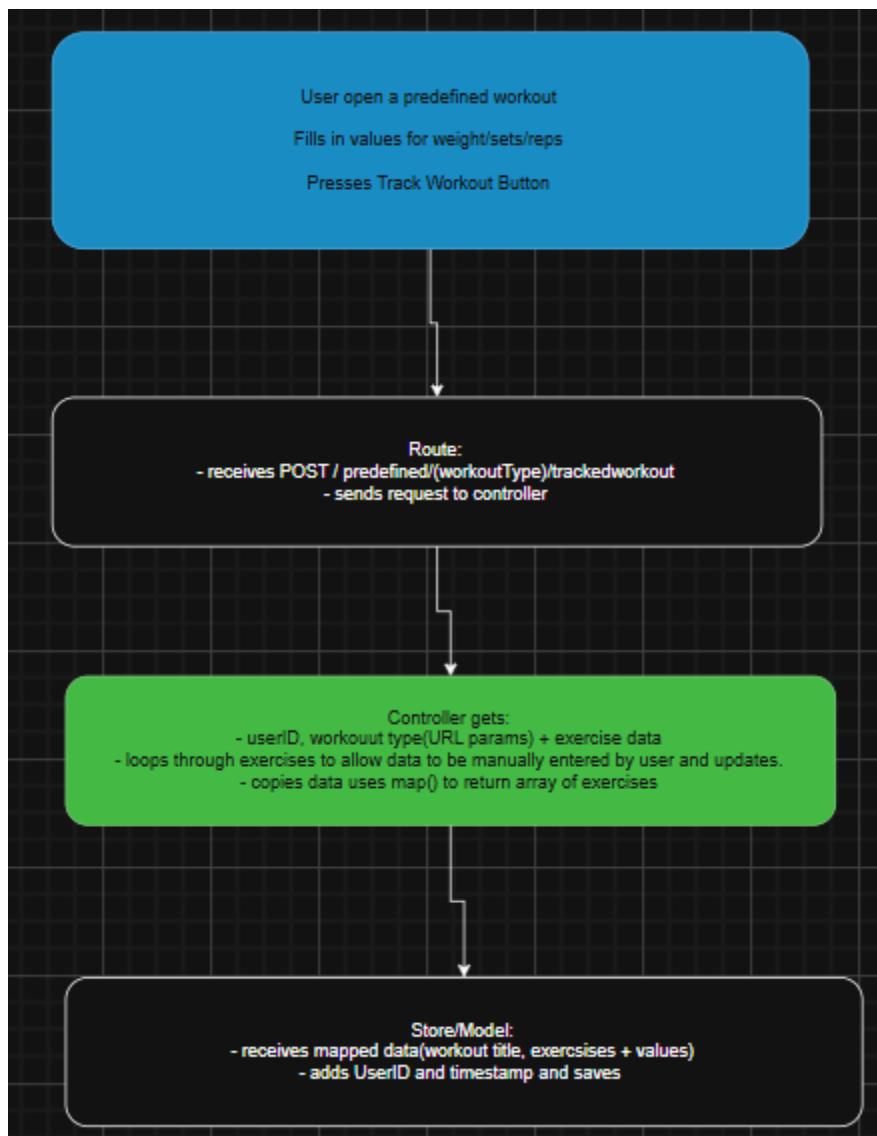
**Figure 3.)** Diagram of full app



**Figure 4.)** Example of adding/deleting a workout in detail.



**Figure 5.)** Example of tracked workout





# Future Development

## 1). Exercise Library Tab

For the exercise tab I would like to add a list of exercises like bench press, shoulder press, push-ups, squats etc. How it might look would be a list of cards with an exercise in each card in alphabetical order. When the user selects a card, a new view opens with the exercise that was chosen. This could be done like how the predefined cards are viewed. Here the user would see a description of the exercise, describing how to do it properly, what equipment to use, an image of the target area, possibly a video showing how to do the exercise properly and a button to add this exercise to a custom workout in dashboard.

A custom workout titled 'arm day', for example, could be made by the user in the dashboard, now if user goes to the exercise library tab, finds the exercise they want, open it there might be a button to add it to a workout. The button could show the list of current custom workouts in the dashboard and the user could pick the correct workout to add it too.

### Filtering/ Search of the exercises in library tab

After creating a list of exercises, it would be good if there was a feature to filter the exercises. This could be done filtering the exercises by body part, e.g. – filter by body type - back, showing suitable exercises, or exercises for certain equipment, e.g. – barbell, matching exercises for this. A search feature could also be developed for simply searching for the exercise.

## 2). Custom Workouts Dashboard

When creating a custom workout in the dashboard, it could also be useful to add a workout type to it as well as the title. For example, if the user types in a new custom workout called 'arm day' there could be a drop-down selection with a list of body parts like 'arms', 'legs', 'back', 'full body' etc. The user could select the body part related to the workout title and add the custom workout to the dashboard. This could be useful for filtering in the tracker tab by workout type.

## 3). The tracker tab

This tab displays the workouts in a list in order it was completed. A future development would be adding a filter feature. The filter feature could find the predefined workouts completed by the title e.g.

- upper body workout, which could display all workouts related to this. This would be useful to see the progression of this type of workout.

Similar for custom workouts, if a body type was added to the custom workout in the dashboard it could be possible to filter custom workouts by type as mentioned above.

#### **4). Timer**

Adding a timer to workouts would be a good feature to have when a workout is opened, before tracking it. The user could fill out the exercises planned for this session and a workout timer button be located here. So the user can time themselves between sets by pressing the timer button, which can be set to 10 – 60 seconds. This is just a timer that doesn't get saved, it's just for using while working out before tracking the exercise.

#### **5). Favourite Button**

Adding a favourite button feature for exercises in the library. These exercises could be saved into a favourite exercises card, so it's easier to find the exercises that the user may regularly use instead of searching/filtering the list of exercises.

#### **6.) Progression**

Maybe by adding a new tab for stats. In this tab there could be a few empty graph/charts set up for weight/sets/ reps. The user could see a drop-down menu seeing a list of exercises to pick from. For example if the user picked something like Bicep curl then then charts/graphs could show the history of biceps curls by the dates they were completed and how much weight/sets/reps were done. The graph/charts could then populate showing the progression of the exercise.